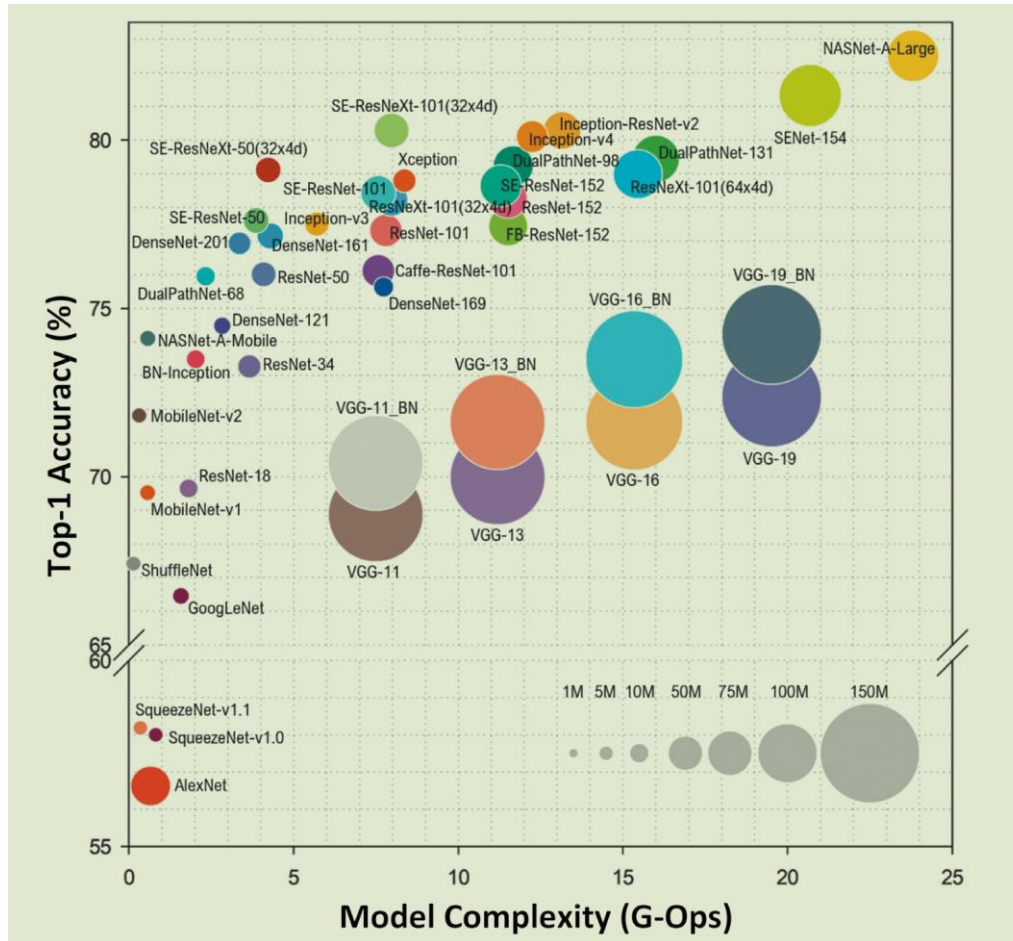

MSML 605

DNN & Hardware

Top-1 accuracy



DNN

- The extensive adoption of DNNs has elevated DNN computation to a distinct category of workload.
- GPUs and CPUs with significant compute parallelism and memory bandwidth are commonly utilized in servers and clouds to speed up DNN tasks.
- However, their high cost and energy consumption make them less than ideal for edge applications.
- Addressing emerging application requirements necessitates the development of domain-specific accelerators tailored for DNN workloads.
- Example: Google's TPU

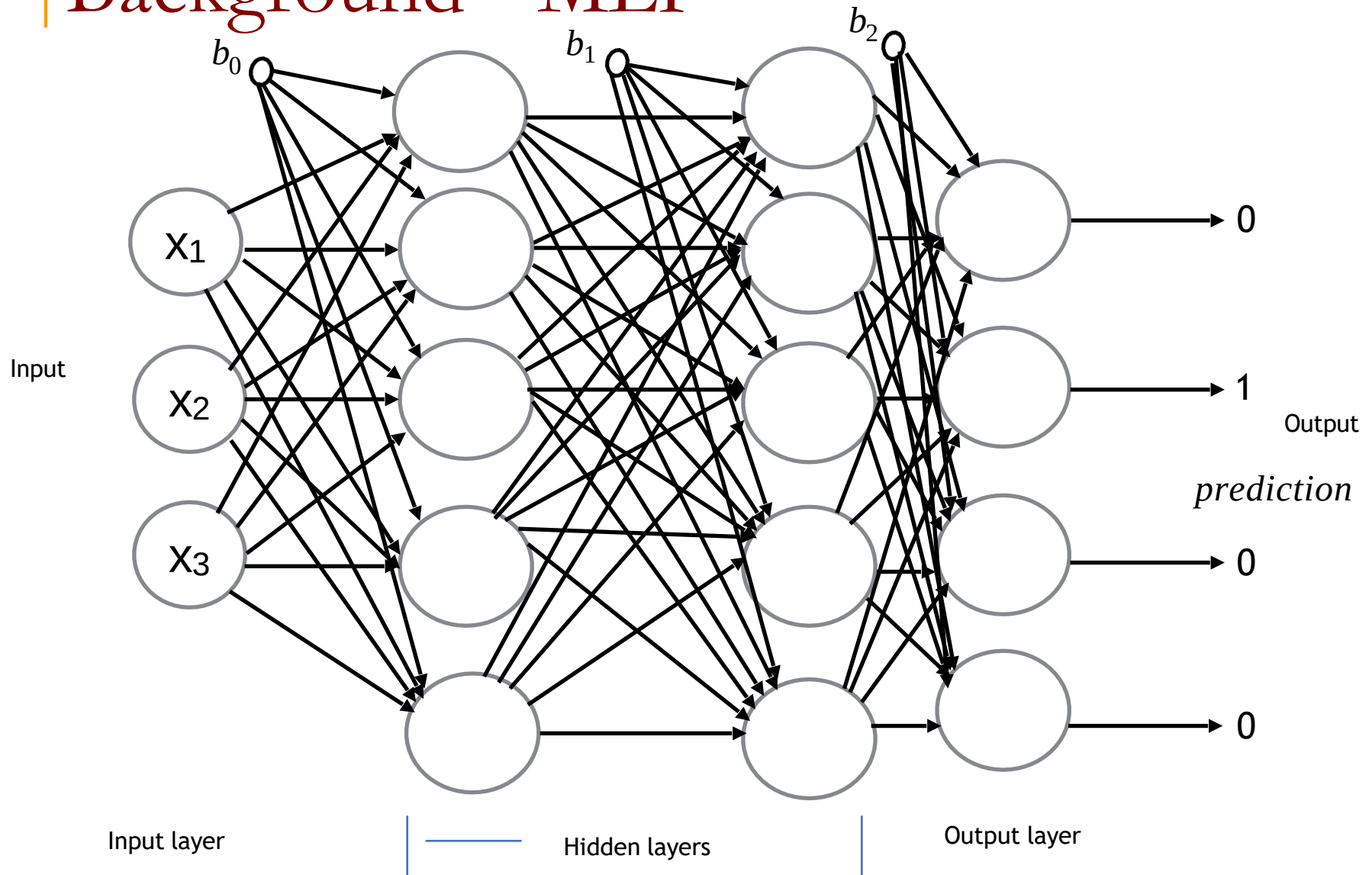
Challenges

- Flexibility: Different DNN models and architectures need different hardware and hence the design needs to be flexible.
 - Efficiency: Both memory and processing needs to be optimized for an advantage over GPUs and CPUs
 - Scalability: The design should incorporate provisions for accommodating larger models that demand increased memory and computational resources, as well as for introducing new iterations of existing models to stay up-to-date and pertinent.
-

Background - Architectures

- Multi-layer perceptron
 - Feedforward full connected network

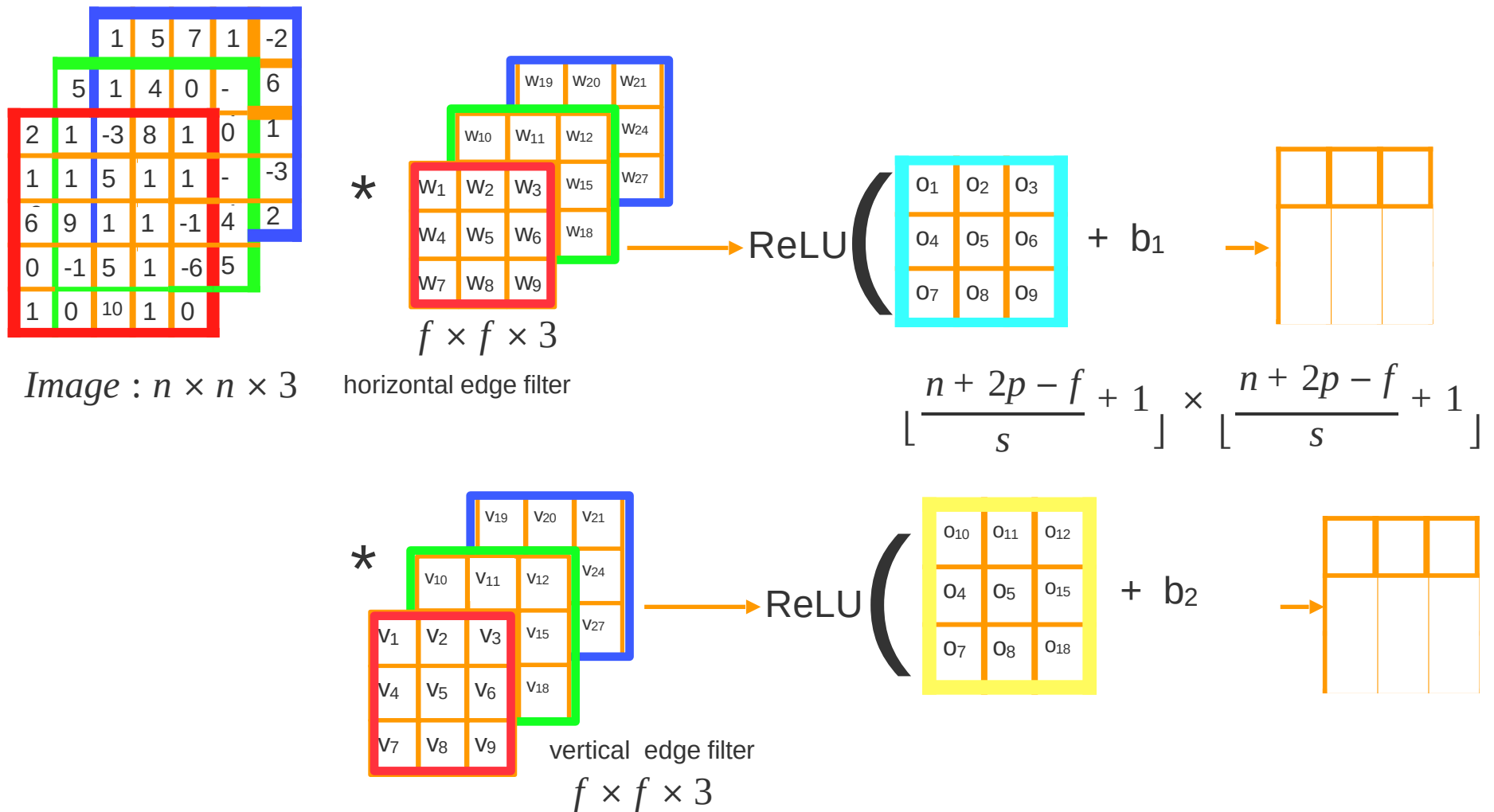
Background - MLP



Convolution Neural Network (CNN)

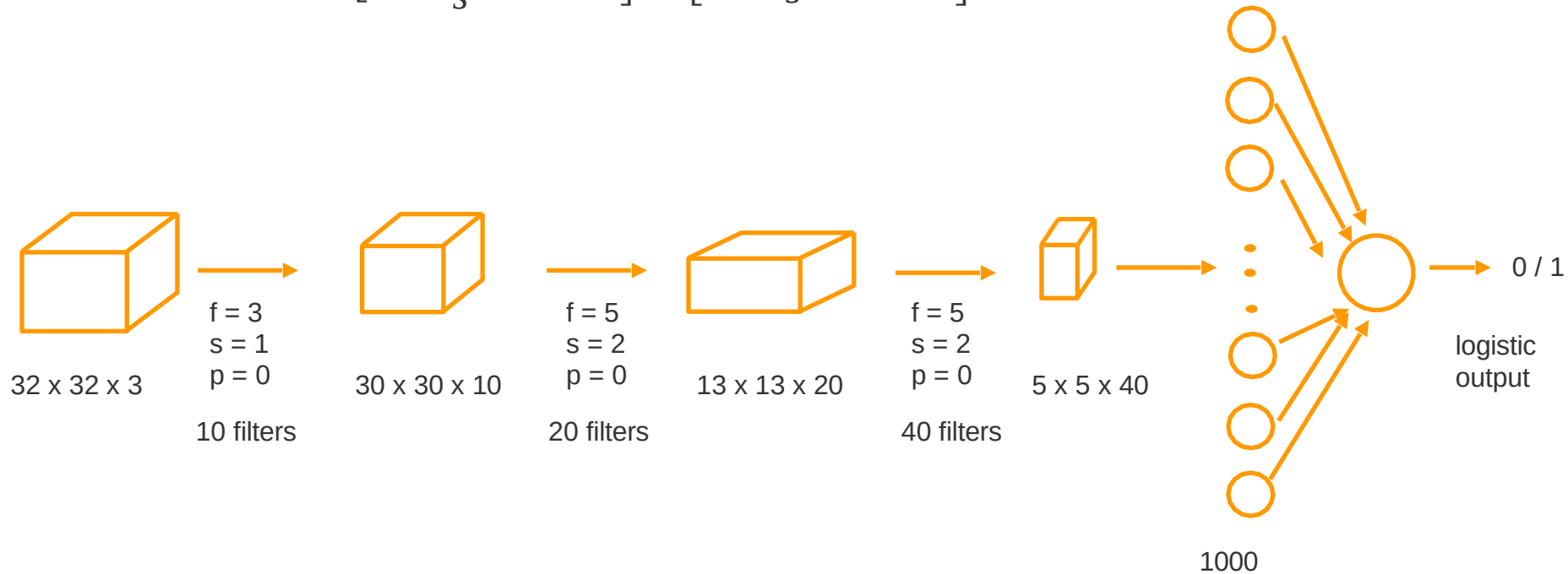
- CNNs are specialized for Computer Vision applications
 - Classification
 - Object Detection
 - Semantic segmentation
-

CNN layer



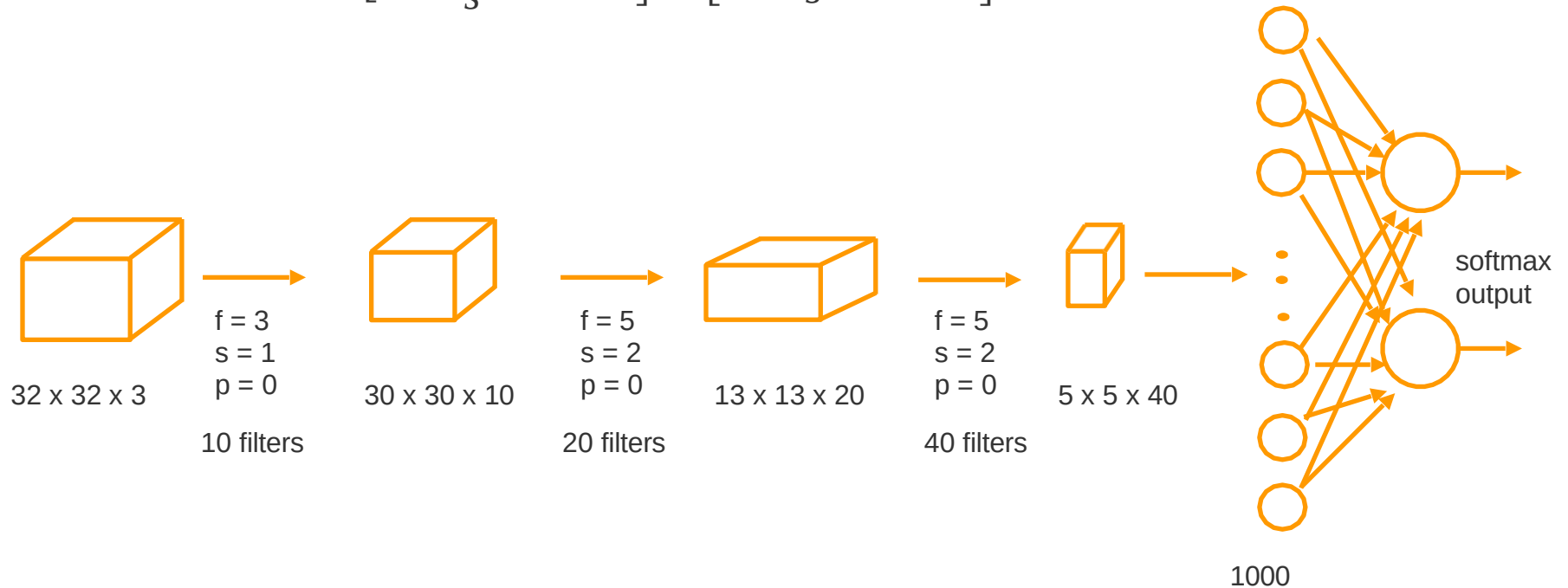
Convolutional Neural Network

$$\left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor$$



Convolutional Neural Network

$$\left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor$$



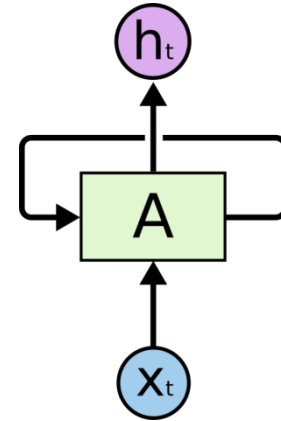
Recurrent Neural Network

- RNNs along with Gated Recurrent Unit (GRU) and Long short term Memory (LSTM) -
 - Used for sequential data - speech, keyword detection and NLP.

Recurrent Neural Networks

Networks with loops

- The output of a layer is used as input for the same (or lower) layer
- Can model dynamics (e.g. in space or time)



Loops are unrolled

- Now a standard feed-forward network with many layers
- Suffers from vanishing gradient problem
- In theory, can learn long term memory, in practice not (Bengio et al, 1994)

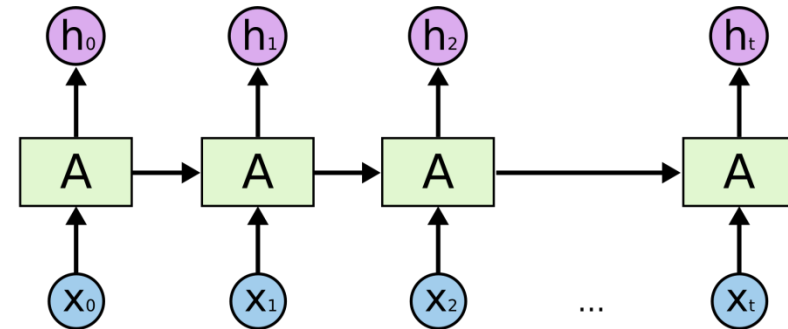


Image credit: Chritopher Olah's blog <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Sepp Hochreiter (1991), Untersuchungen zu dynamischen neuronalen Netzen, Diploma thesis. Institut f. Informatik, Technische Univ. Munich. Advisor: J. Schmidhuber.

Y. Bengio, P. Simard, P. Frasconi. Learning Long-Term Dependencies with Gradient Descent is Difficult. In TNN 1994.

RNN Architectures

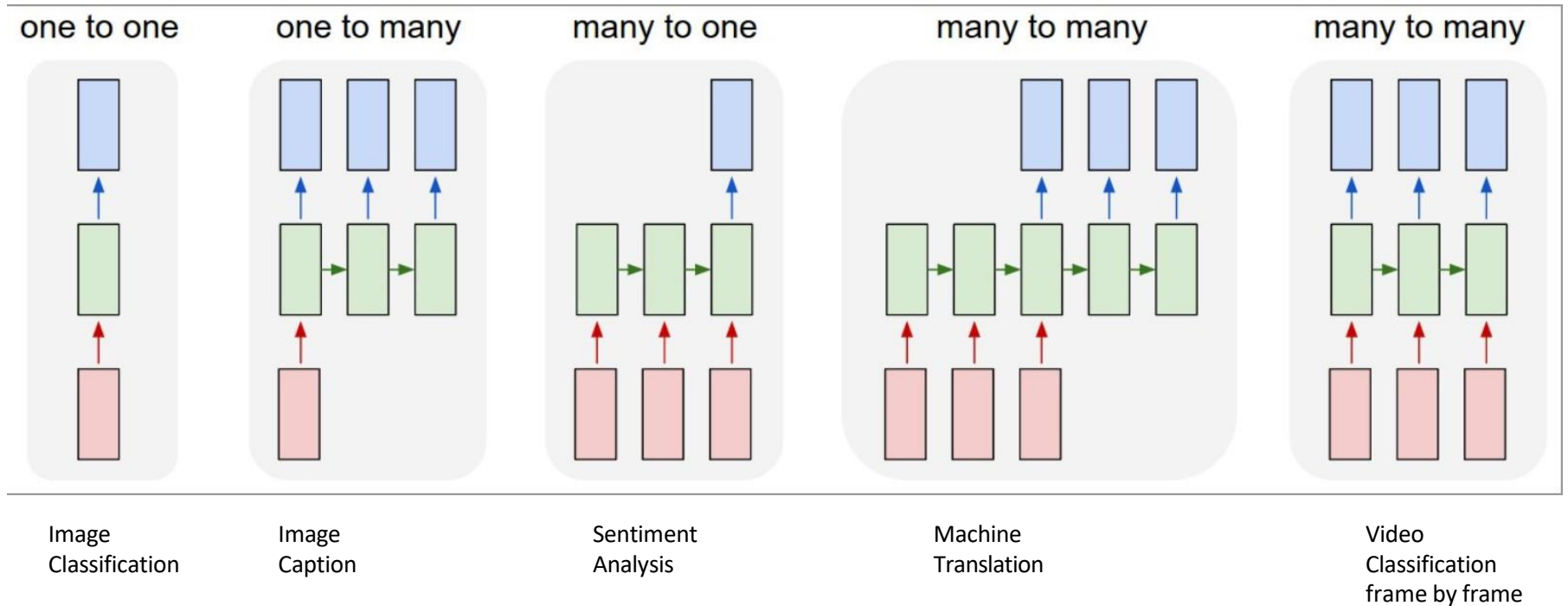
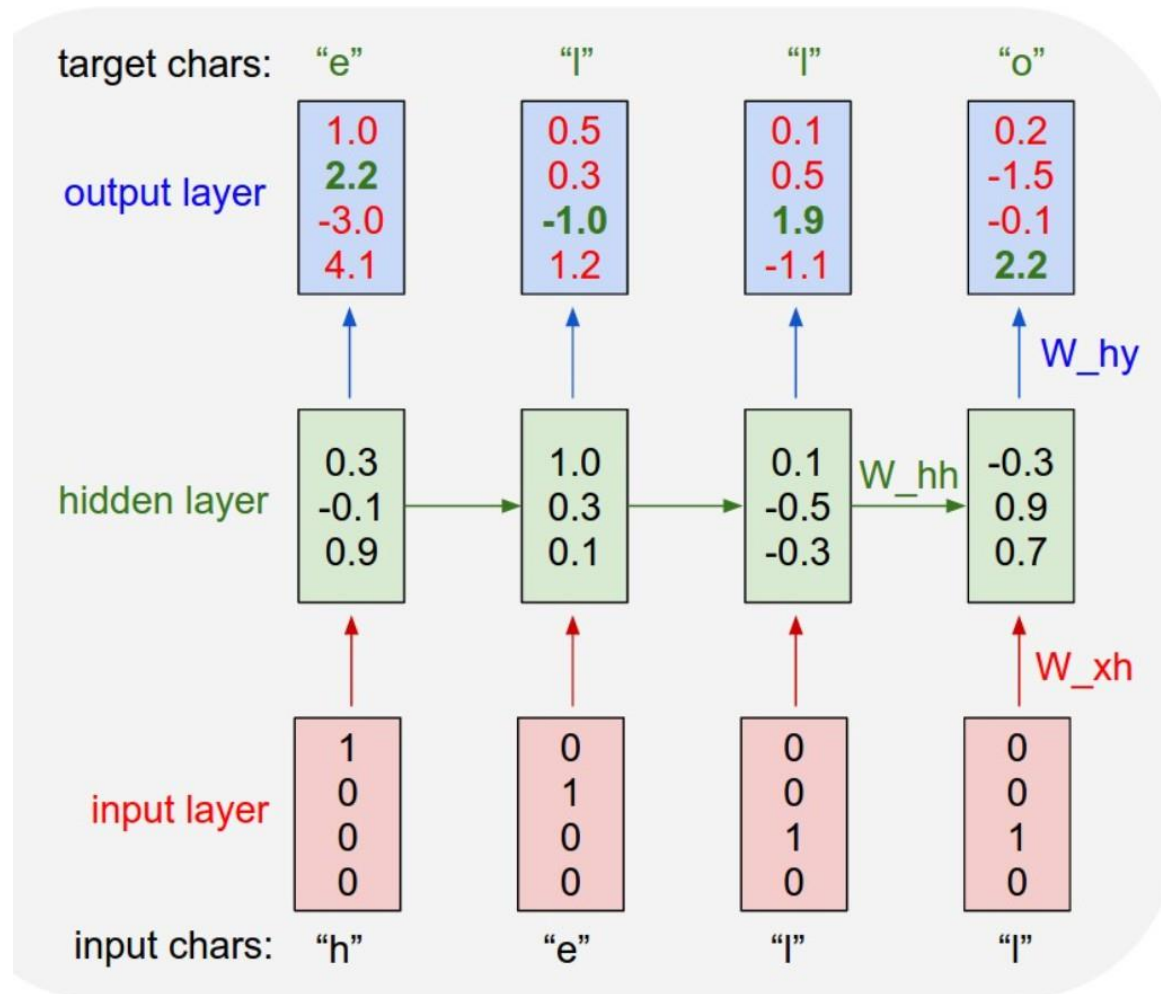
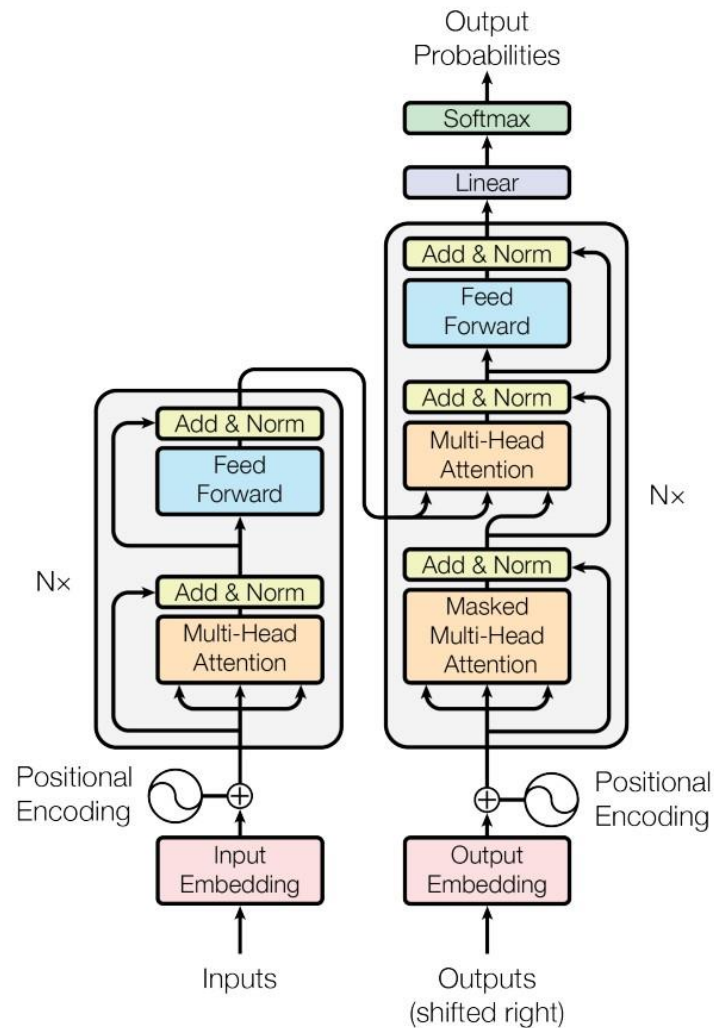


Image credit: Andrej Karpathy's blog, <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Example RNN

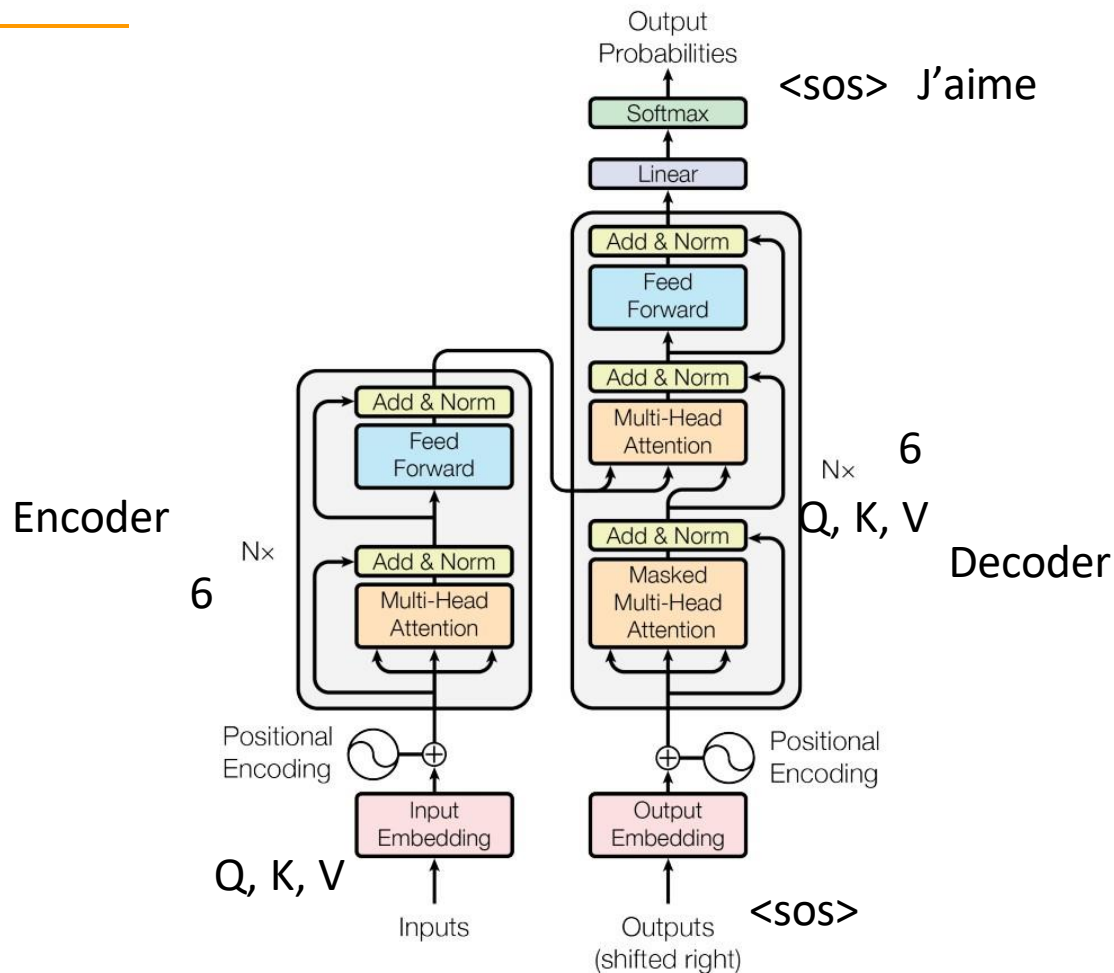


Transformers



<https://arxiv.org/abs/1706.03762>

Transformer

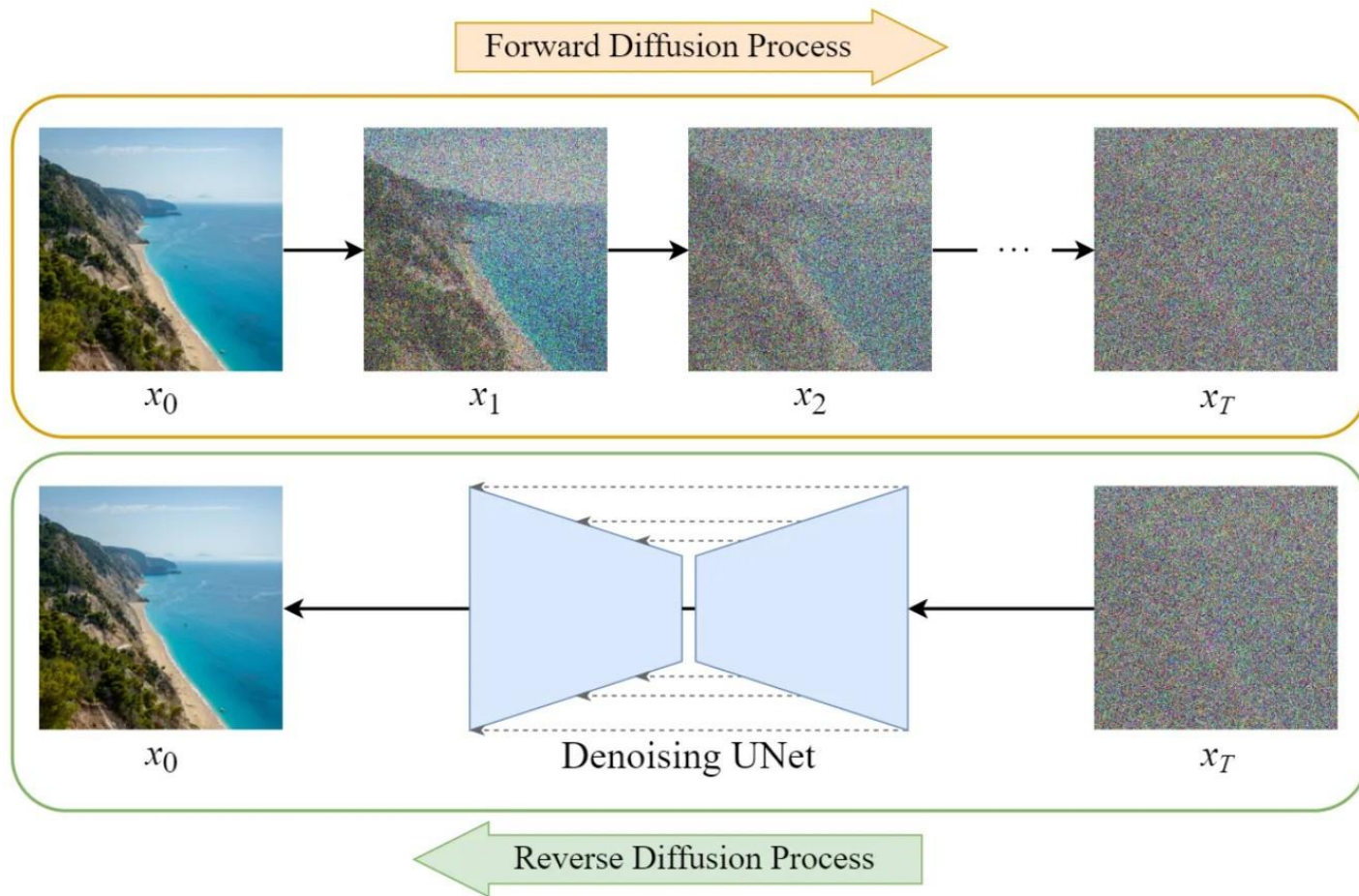


<sos> x_1 x_2 x_3 x_4 <eos>

- Example:
Input (English): "I like mountain climbing."

Output (French): "J'aime l'escalade en montagne."

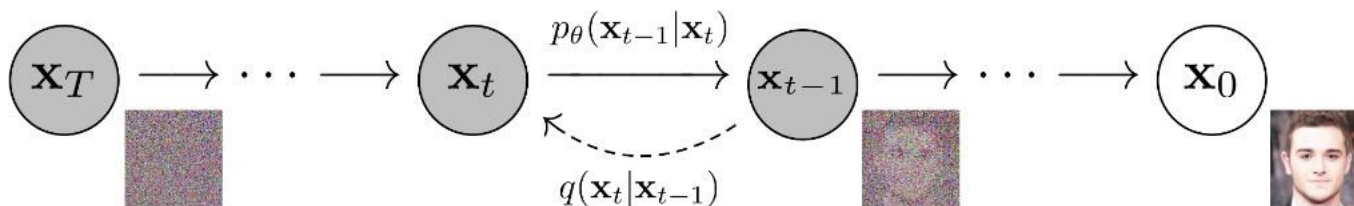
Diffusion Networks



Overview of the Diffusion Model

Diffusion Networks

- Diffusion models generate samples by gradually removing noise from a signal,
- Their training objective can be expressed as a reweighted variational lower-bound (<https://arxiv.org/abs/2006.11239>).



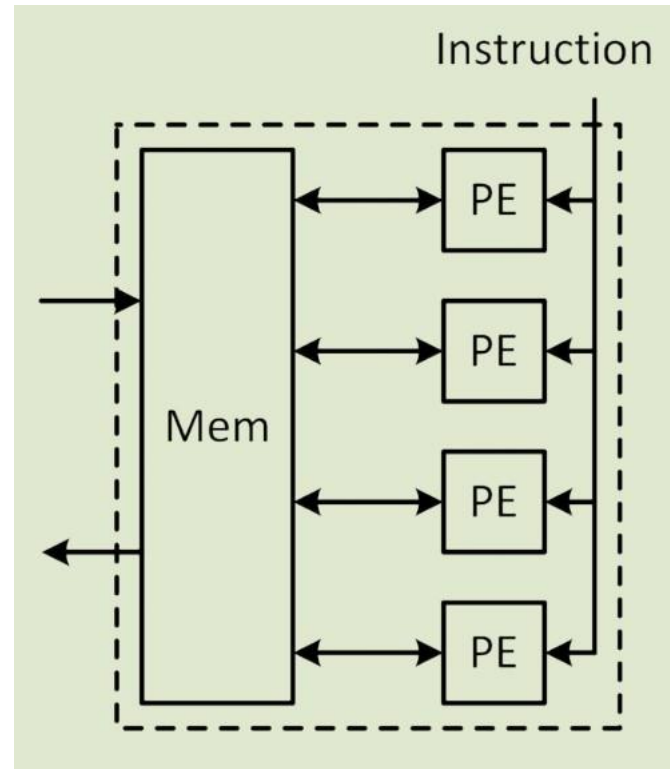
Diffusion probabilistic models are parameterized Markov chains trained to gradually denoise data. We estimate parameters of the generative process p .

<https://arxiv.org/pdf/2006.11239.pdf>

DNN Architectures - VMM & MMM

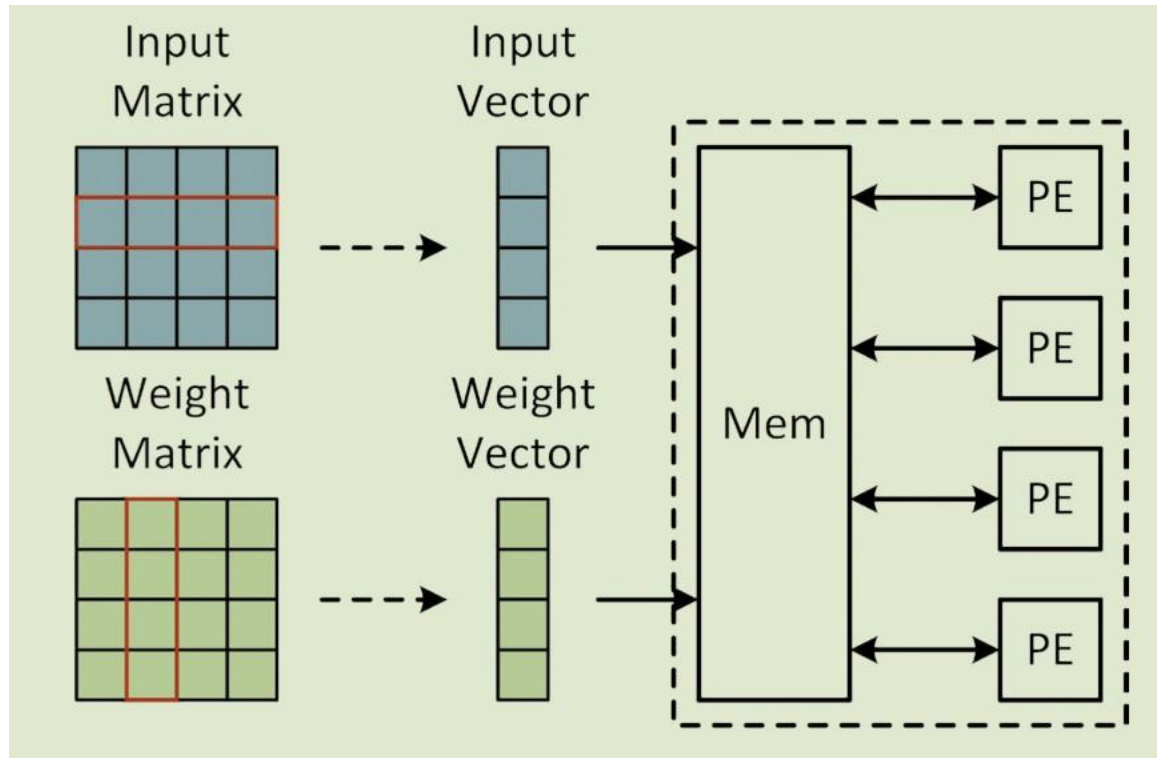
- Single instruction Multiple Data (SIMD)
- Systolic Array

SIMD Architecture - Single instruction

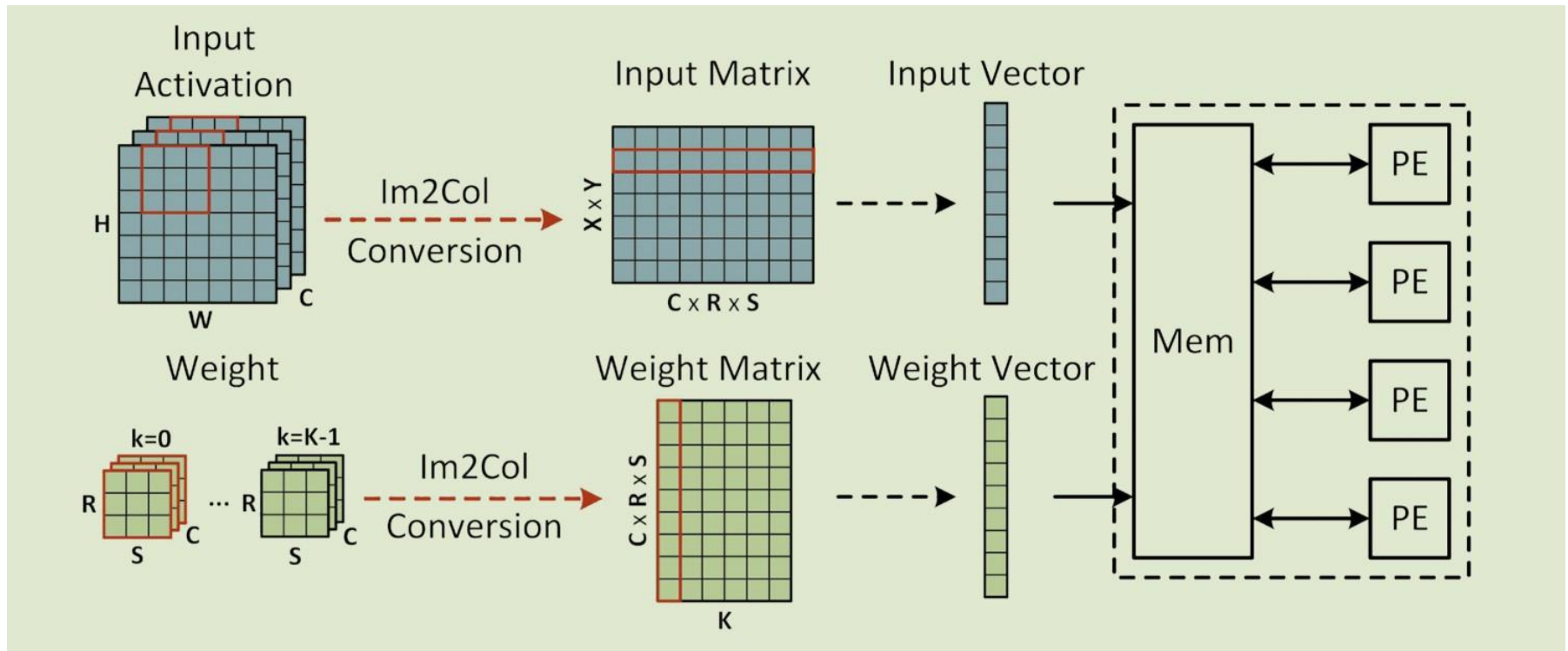


PE- Parallel processing
Elements

SIMD Architecture - Matrix to matrix multiplication

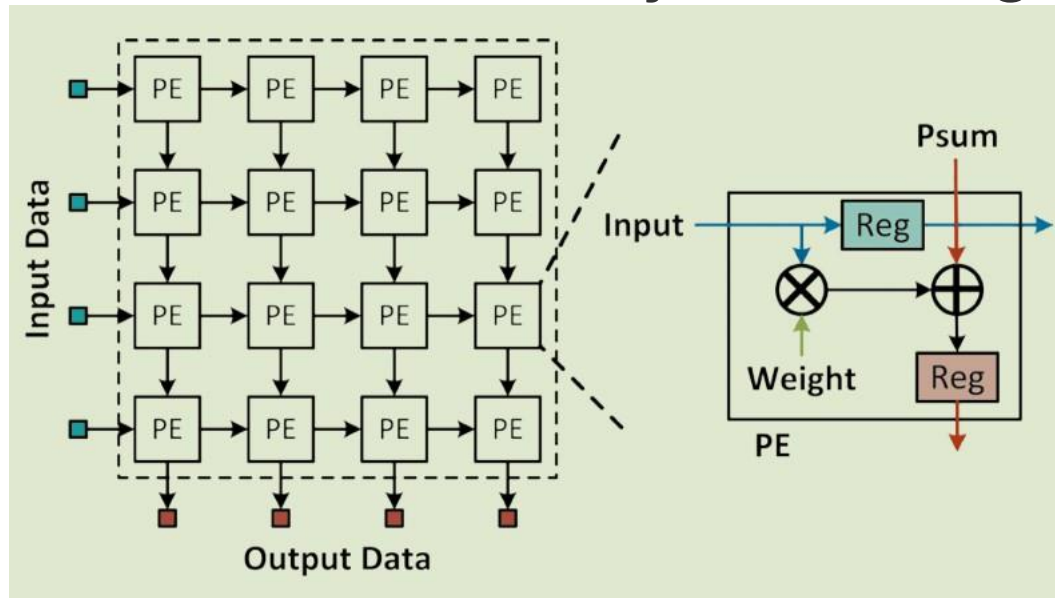


SIMD Architecture - Conv



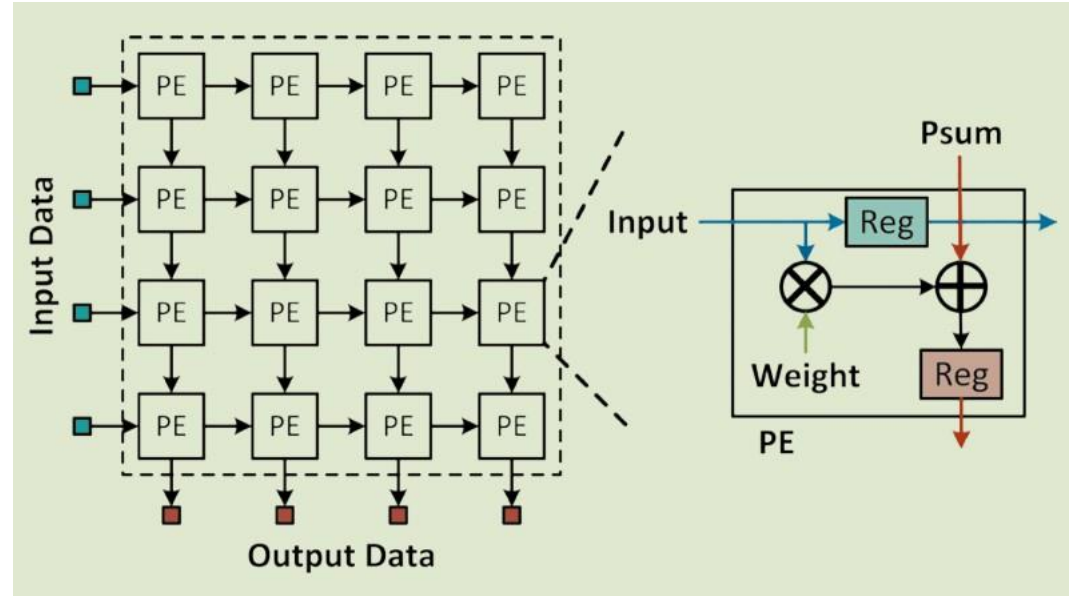
Systolic Array Architecture

- A systolic array comprises of a structured 2D array of processing elements (PEs), with each PE linked to its adjacent neighbors.



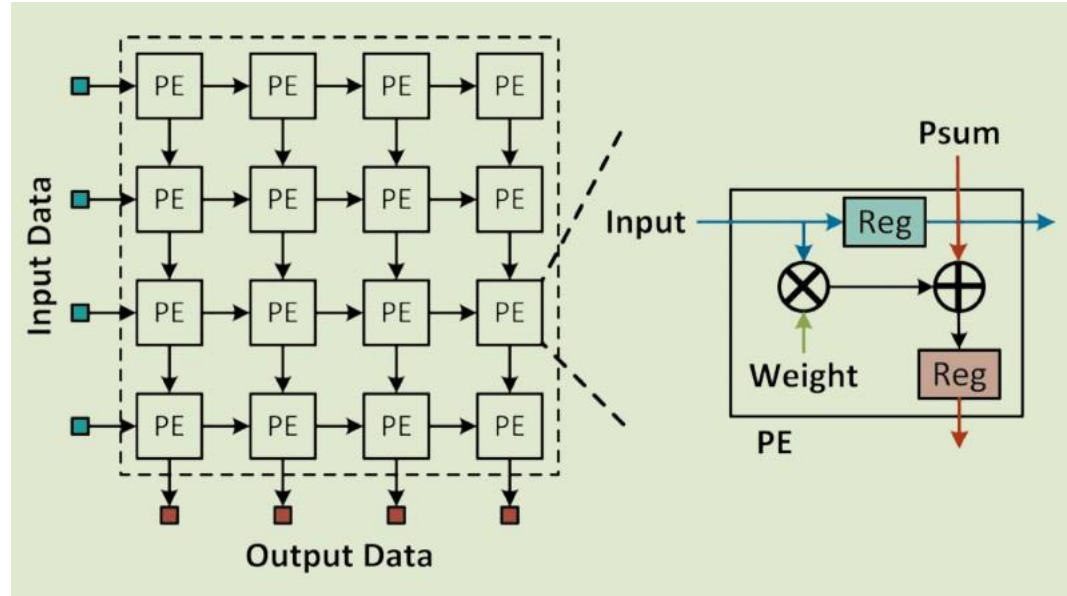
Systolic Array Architecture

- The inputs are routed to the processing elements (PEs) through a border of the PE array, such as the leftmost column.
- Subsequently, the interim outcomes are transmitted across the PEs, horizontally to the right and vertically downward.
- Finally, the output are sent out through another end of the PE array, e.g., bottom row.



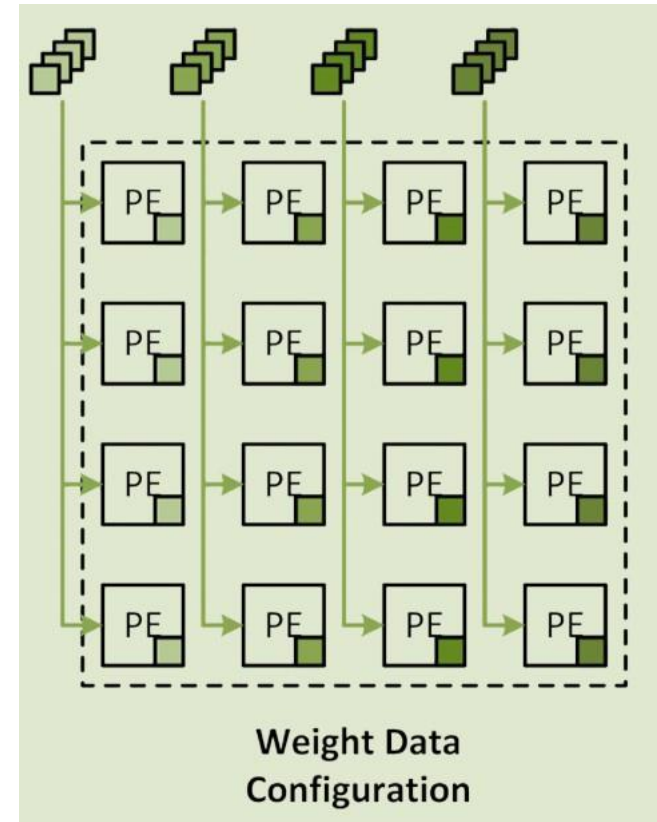
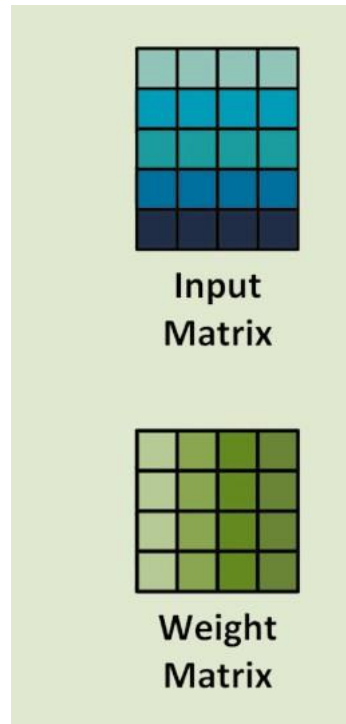
Systolic Array Data flow

- A PE is commonly designed with a multiplier to compute the product of an incoming input and a cached weight value,
- and an adder to sum the computed product and an incoming partial sum (psum).
- The updated psum is sent vertically to the next PE down and the input is propagated horizontally to the next PE on the right.



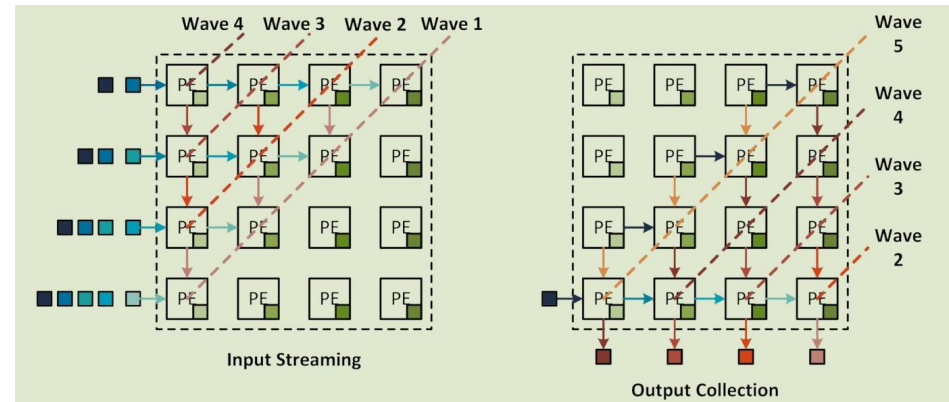
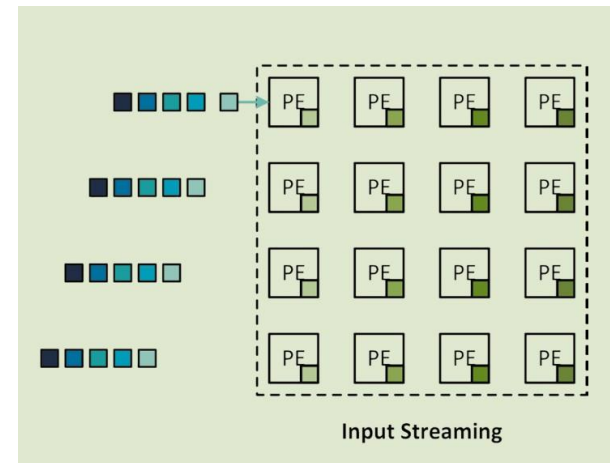
Systolic Array MMM

- Before performing an MMM operation on a systolic array, the weights are initially loaded onto the array.
- Each vector is streamed to and stored in the corresponding column of the PE array column



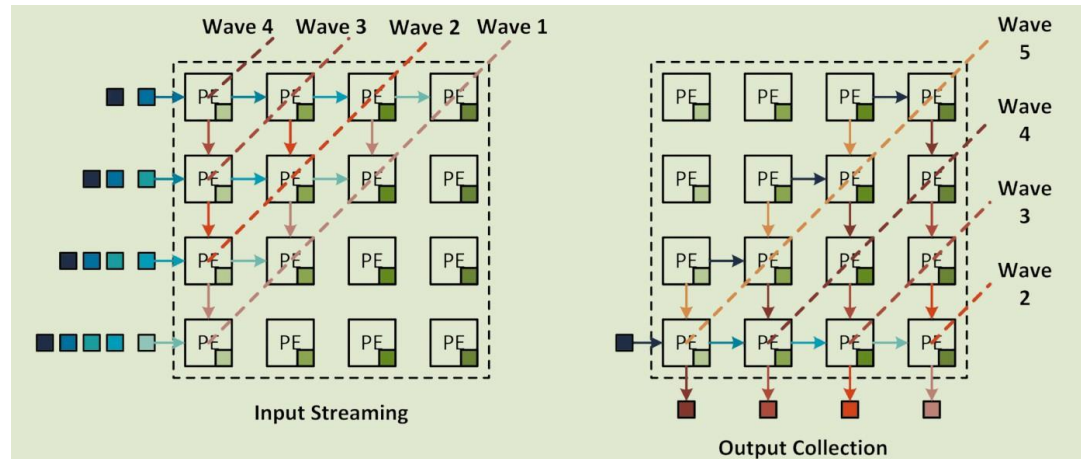
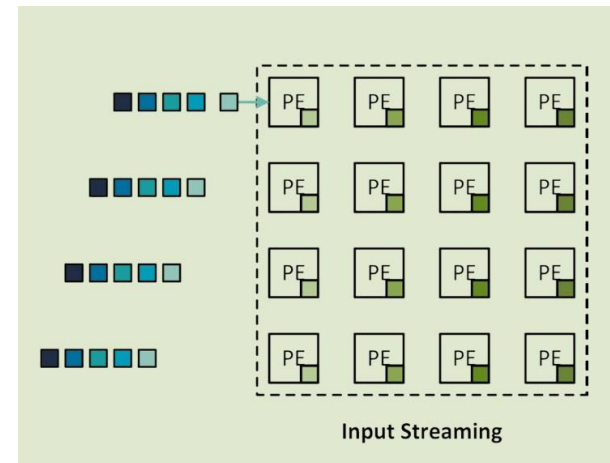
Systolic Array MMM

- The input matrix is split into row vectors that are streamed sequentially to the PE array
- The inputs propagate from left to right, passing through one PE in a clock cycle.
- When an input enters a PE, the PE computes the product between the input and the cached weight, and sums the product with the psum that enters from top.



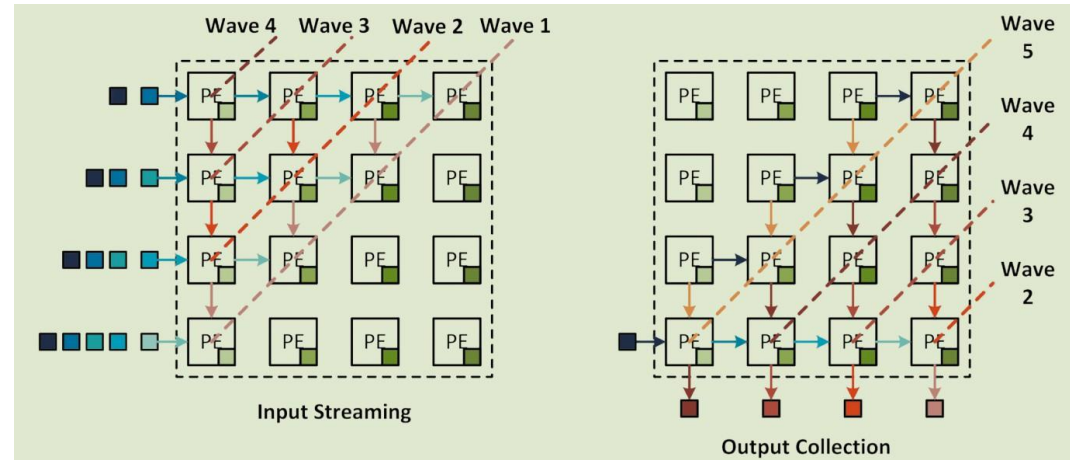
Systolic Array MMM

- When an input enters a PE, the PE computes the product between the input and the cached weight, and sums the product with the psum that enters from top.
- Following the computation, the PE passes the input to the next PE on the right and the updated psum to the next PE down.
- Note that the inputs to the rows of PE must be arranged with a one cycle delay from one row to the next to ensure that the correct psum accumulation.



Systolic Array MMM

- Note that the inputs to the rows of PE must be arranged with a one cycle delay from one row to the next to ensure that the correct psum accumulation.
- Data move through the systolic array in waves.
- The wavefront propagates diagonally across the systolic array.
- The outputs are collected from the bottom row of PEs.



SIMD vs Systolic Array

- In contrast to a SIMD (Single Instruction, Multiple Data) array, a systolic array is preconfigured with a specific dataflow pattern, making it less adaptable to different computational tasks.
 - A sizable systolic array offers greater computational capacity, yet it could experience reduced efficiency if operations fail to fully engage the entire array.
 - Another downside of a large-size systolic array is its extended latency, which can impede overall performance.
-

SIMD vs Systolic Array

- A SIMD array can be in the form of a 1D PE array to support VMM operations, and it can also be scaled up to a 2D PE array to support MMM operations.
 - In terms of data movement, a SIMD array needs to access memory to feed all its PEs, whereas
 - a systolic array can rely on neighboring PE connections to reduce the bandwidth requirement and the number of memory accesses.
 - A systolic array has a lower control overhead and can be easily scaled up.
-

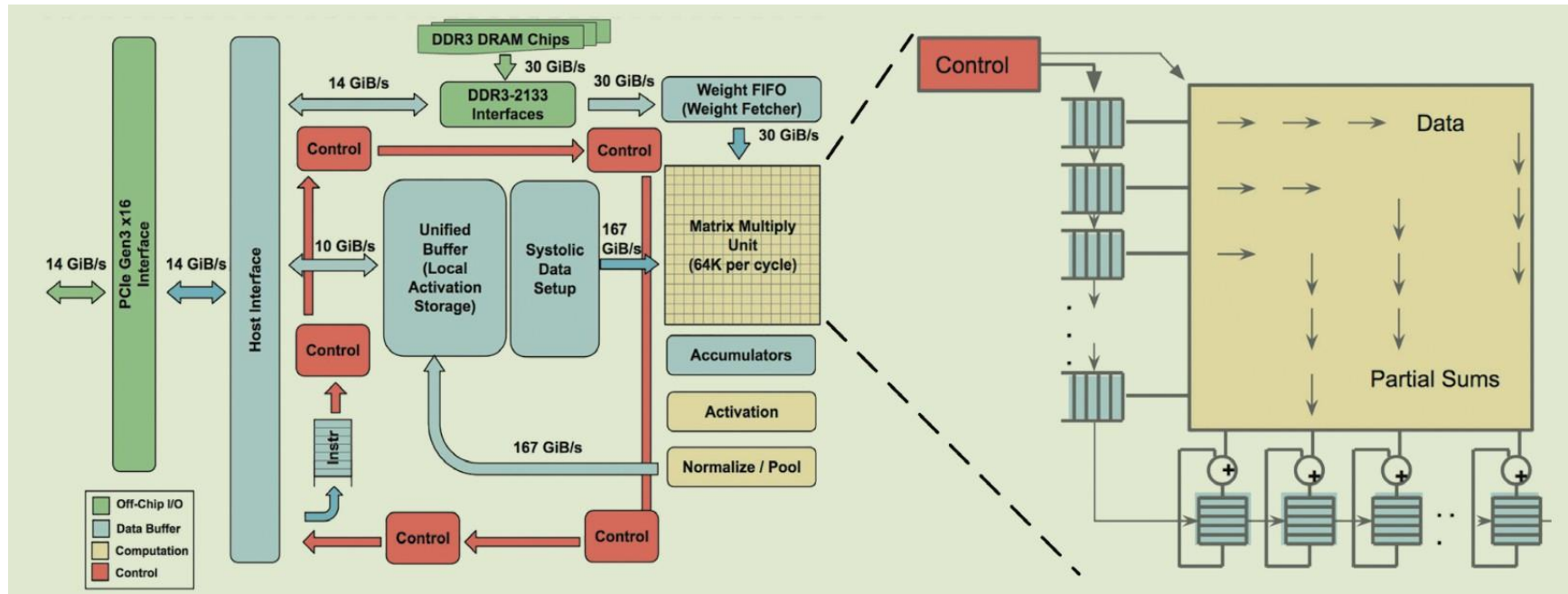
SIMD vs Systolic Array

- A systolic array provides a higher compute density than a SIMD array.
 - a systolic array is often designed for a fixed computation, whereas a SIMD array is more flexible to support a wide range of operations.
 - The higher flexibility of a SIMD array over a systolic array leads to a higher hardware utilization, e.g., in flexibly handling inputs of various shapes.
-

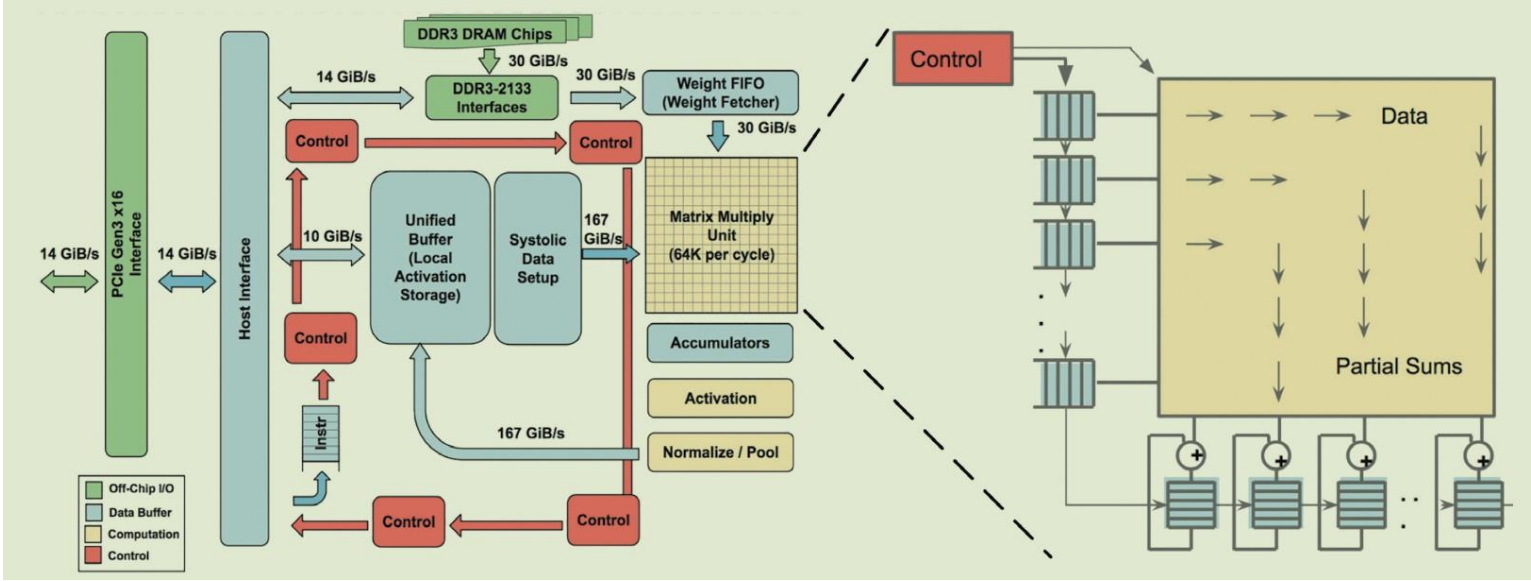
TPU

- TPU is an example of systolic array.
- TPU is designed with a matrix multiplication unit (MMU) that consists of a systolic array of 256×256 PEs
-

TPU



TPU

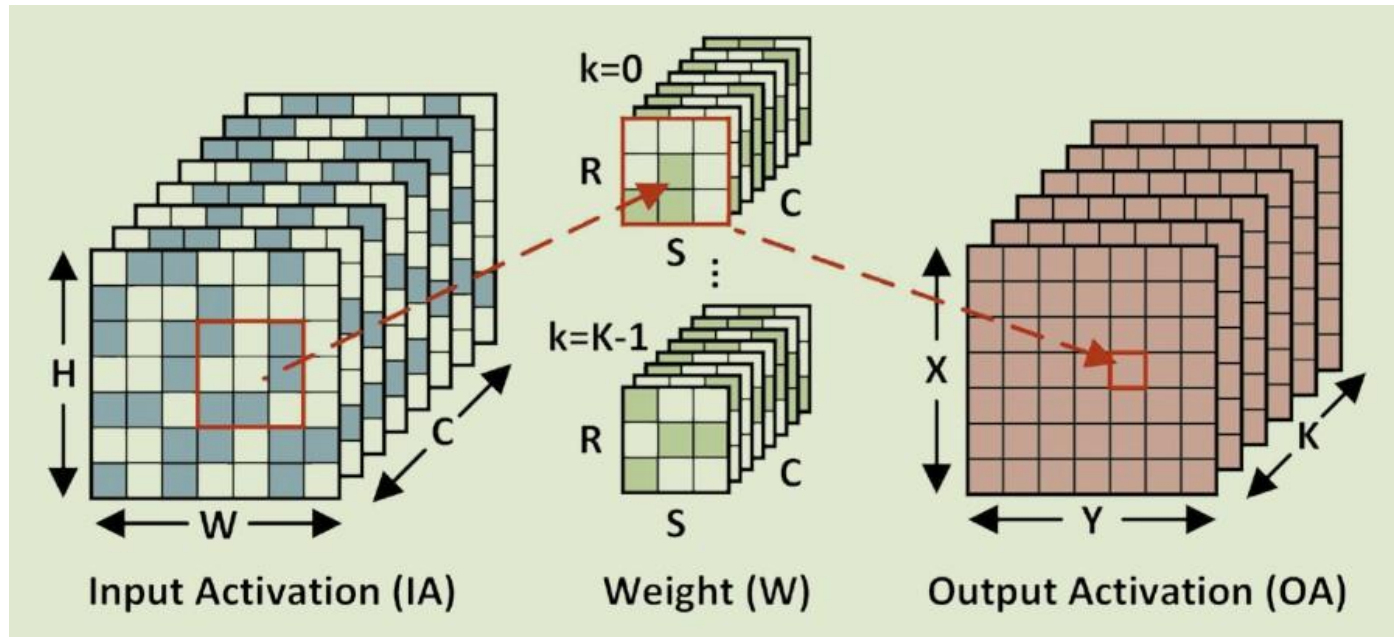


- The weight data are loaded from the weight FIFO into the MMU, and a systolic data setup module organizes the input data to ensure proper accumulation of the psums in the MMU.
- The MMU operates similarly to the description above where inputs are streamed in from the left to the right, and the psum accumulation happens vertically across columns.

Sparse Architecture - Sparsity in NN

- Network sparsity arises from both the model's weights (Ws) and input activations (IAs).
- Techniques like network pruning and other methods of sparsification can be employed to eliminate a significant number of weights from a model, resulting in only a minor decrease in inference accuracy.
- Regarding input activations, certain frequently utilized operators like the rectifier linear unit (ReLU) can constrain all negative activations to zero, leading to sparsity in output activations (OAs), which then serve as input activations (IAs) for the subsequent layer.

CONV computation



Benefits of exploiting sparsity

- Data sparsity can be exploited to save power.
 - Second, data sparsity can be used to reduce off-chip memory storage and bandwidth usage.
 - data sparsity can be used to reduce latency by skipping the ineffectual computation.
-

Sparse Compression Format

- Sparse compression formats are utilized to compactly store sparse data, aiming to conserve storage space.
 - A compressed format contains only nonzero data values and metadata to hold the information for locating the positions of nonzero values in the uncompressed vectors and matrices.
-

Common Sparse compression formats

- Coordinate List (COO): In the COO format, nonzero data are stored along with their absolute indices in the original uncompressed vector or matrix

Uncompressed Format	COO Format																																		
<table><tr><td>0</td><td>a</td><td>0</td><td>0</td></tr><tr><td>0</td><td>b</td><td>c</td><td>0</td></tr><tr><td>0</td><td>0</td><td>d</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>e</td></tr></table>	0	a	0	0	0	b	c	0	0	0	d	0	0	0	0	e	<table><tr><td>Value</td><td>a</td><td>b</td><td>c</td><td>d</td><td>e</td></tr><tr><td>Row Index</td><td>0</td><td>1</td><td>1</td><td>2</td><td>3</td></tr><tr><td>Col Index</td><td>1</td><td>1</td><td>2</td><td>2</td><td>3</td></tr></table>	Value	a	b	c	d	e	Row Index	0	1	1	2	3	Col Index	1	1	2	2	3
0	a	0	0																																
0	b	c	0																																
0	0	d	0																																
0	0	0	e																																
Value	a	b	c	d	e																														
Row Index	0	1	1	2	3																														
Col Index	1	1	2	2	3																														

- The advantage of COO is its low decoding complexity, since the row and column indices can be directly used to locate the positions of the nonzero data in the uncompressed vector or matrix.
- However, the row and column indices may require significant amount of storage overhead which makes COO less efficient for data of medium or low sparsity.